

Domain Separation — `admin.osteopathie.eu` VS `teacher.osteopathie.eu`

Context: The IAO Belgium frontend (`IAO-Belgium-Frontend`) is a single React SPA that today serves **both** admin and teacher from one deployment (previously `backoffice.osteopathie.eu`). The client now wants two **separate-brand, separate-access** portals:

- `admin.osteopathie.eu` — admin only
- `teacher.osteopathie.eu` — teacher only

Date: 2026-06-24

Branch: `test` (UAT)

Companion: [FRONTEND_CODE_AUDIT.md](#)

1. The question, restated

"The codebase technically handles both roles already (separate `adminRoutes` / `teacherRoutes` , separate page trees). But at the **domain** level I don't know what to do. Should I clone it and run two nginx setups? If I do that, a teacher who logs in on `teacher.osteopathie.eu` could still end up using the admin view — which is weird. What's the best practice?"

Short answer: Do **not** clone the repo. Keep **one** codebase, build it **once**, serve it on **two hostnames**, and make the app **host-aware** so each domain only exposes its own portal. But the part that actually *prevents* cross-access is **server-side role enforcement on the backend** — not anything the frontend does.

2. Why your instinct is correct — and what actually causes the "weird" cross-access

Today the only thing separating admin from teacher in the browser is **React code**:

- [src/layouts/AdminTeacherLayout.jsx:24-42](#) redirects based on `path.startsWith("/admin")` VS `role` .
- [src/layouts/admin-teacher-layout.jsx:31](#) switches the sidebar with `const isTeacher = user?.role === "teacher"` .

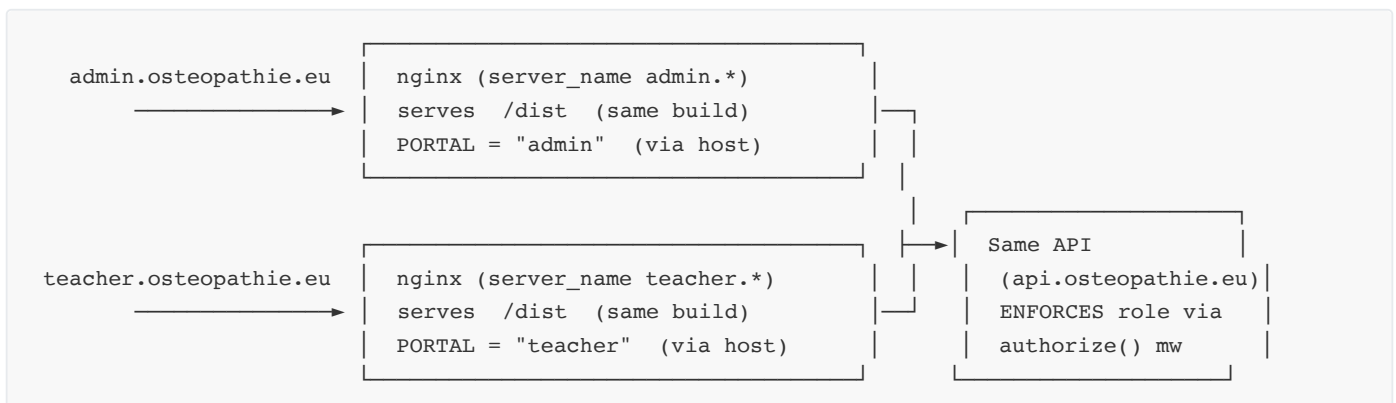
That is **UX gating, not security**. A determined user can bypass it in devtools or by calling the API directly. So:

Cloning + two nginx, by itself, does NOT fix the cross-access problem. Both clones would talk to the same API with the same `VITE_APP_API_KEY` and the same JWT. If the backend doesn't check roles, a teacher token can still drive admin actions regardless of which domain the page was served from.

The real boundary is the backend. The frontend work below makes the separation *clean and branded*; the backend work makes it *enforced*.

3. Recommended architecture: one codebase, two hosts, host-aware app

3.1 Picture



One repo. One `npm run build`. The app decides which portal it is at **runtime from** `window.location.hostname` (no separate build needed — though you *can* use a build-time env var if you prefer two artifacts).

3.2 The three changes on the frontend

(A) Detect the portal from the host. Add `src/config/portal.js`:

```
// Resolve which portal this domain is, from the hostname.
const HOST = window.location.hostname;
export const PORTAL =
  HOST.startsWith("admin.") ? "admin" :
  HOST.startsWith("teacher.") ? "teacher" :
  // local dev / fallback: allow both (current behaviour)
  "both";
```

(Optionally back this with `import.meta.env.VITE_PORTAL` so local dev can force a portal.)

(B) Mount only that portal's route tree. In `src/router/Router.jsx`, the protected children currently merge **both** trees:

```
const protectedRouteObjects = [
  ...teacherRoutes.map(...),
  ...adminRoutes.map(...),
];
```

Make it portal-aware:

```
import { PORTAL } from "../config/portal";
const protectedRouteObjects = [
  ...(PORTAL !== "admin" ? teacherRoutes : []).map(...),
  ...(PORTAL !== "teacher" ? adminRoutes : []).map(...),
];
```

Now `admin.osteopathie.eu` literally does not ship teacher routes, and vice-versa. (Code-split per portal later for smaller bundles — see audit §6.)

(C) Reject the wrong role at login, per host. This is what kills the "weird" scenario. In `src/pages/Login.jsx:82-105` and the auto-redirect `:51-59`, after `verifyotp` check the role against `PORTAL`:

```
import { PORTAL } from "@config/portal";
// after login:
if (PORTAL === "admin" && userRole !== "admin") {
  toast.error("This is the admin portal. Teachers, please use teacher.osteopathie.eu");
  await useAuthStore.getState().logout();
  return;
}
if (PORTAL === "teacher" && userRole !== "teacher") {
  toast.error("This is the teacher portal. Admins, please use admin.osteopathie.eu");
  await useAuthStore.getState().logout();
  return;
}
// else navigate to the portal's dashboard
```

So even if an admin enters their OTP on `teacher.osteopathie.eu`, they're logged straight back out with a redirect hint. No admin view ever renders on the teacher domain.

(D) Brand per portal. Use `PORTAL` to pick logo / theme / title:

```
const brand = PORTAL === "teacher"
  ? { logo: teacherLogo, name: "IAO Teacher Portal", accent: "..."}
  : { logo: adminLogo, name: "IAO Admin Portal", accent: "..."};
```

Apply to `Login.jsx`, the layout header, and `index.html` title. The two domains now *look* like separate products from one codebase.

You can also **drop** `LoginSelection.jsx` on the branded domains — each portal goes straight to its own login. Keep the chooser only if you still want a neutral landing page somewhere.

3.3 nginx (one bundle, two server blocks)

```
server {
  server_name admin.osteopathie.eu;
  root /var/www/iao-frontend/dist;          # same build artifact
  location / { try_files $uri $uri/ /index.html; } # SPA fallback (replaces vercel.json rewrite)
  # TLS via wildcard *.osteopathie.eu or per-host cert
}
server {
  server_name teacher.osteopathie.eu;
  root /var/www/iao-frontend/dist;          # SAME build
  location / { try_files $uri $uri/ /index.html; }
}
```

Because the app reads the host at runtime, **both server blocks point at the same `dist/`**. No fork. (If you adopt the `VITE_PORTAL` build-time variant instead, build twice into `dist-admin/` and `dist-teacher/` and point each block at its own folder.)

Note: `vercel.json`'s rewrite is the Vercel equivalent of the `try_files` SPA fallback. Moving to nginx, you must recreate it as shown — otherwise deep links (e.g. `/admin/intake/123`) 404 on refresh.

3.4 The mandatory backend half (the actual security)

None of the above is a security boundary until the backend enforces roles. Two pieces:

1. **`authorize()` middleware** so admin endpoints reject teacher tokens and vice-versa (this is the real fix and belongs in the backend repo's plan).

2. **Optional — per-portal audience claim.** Stamp `aud: "admin-portal" / "teacher-portal"` into the JWT at login and verify it. Then an admin token literally won't be accepted on teacher-portal API calls. Defense in depth.

4. Options compared

Option	Cross-access prevented?	Maintenance	Verdict
Clone repo + 2 nginx, no backend RBAC	❌ No — teacher token still drives admin API directly	2× everything (every fix twice)	Don't. Exactly the "weird" scenario you fear, and double the work.
One codebase, host-aware app + backend <code>authorize()</code> 	✅ Yes (backend enforces; frontend stays clean & branded)	One repo, one build	Recommended.
Fully separate apps + separate DB / tenancy	✅ Yes	Highest (two products)	Overkill — admin & teacher share the same data model and API here.

5. Why NOT to clone (summary)

- **Doubles maintenance forever** — every bugfix, dependency bump, and audit (including the XSS fix in the audit doc) must be applied twice and kept in sync.
- **Doesn't add security** — both clones hit the same API with the same key/JWT; the boundary you actually need is server-side.
- **The codebase already separates the two cleanly** (`adminRoutes` vs `teacherRoutes`, separate page trees, role-switched layout) — cloning throws away that and gains nothing.

6. Implementation checklist

Frontend (this repo):

- ☐ Add `src/config/portal.js` (host → `"admin" | "teacher" | "both"`).
- ☐ Make `Router.jsx` mount only the active portal's routes.
- ☐ Reject wrong-role login per portal in `Login.jsx` (auto-logout + redirect hint).
- ☐ Per-portal branding (logo / title / theme) driven by `PORTAL`.
- ☐ Env-var the student portal URL (audit §4.2) so the rebrand isn't hardcoded.
- ☐ (Optional) Code-split admin vs teacher bundles with `React.lazy`.

Infra:

- ☐ Two nginx `server_name` blocks → same `dist/`, each with `try_files ... /index.html`.
- ☐ DNS `admin.` and `teacher.` A/CNAME records.
- ☐ TLS: wildcard `*.osteopathie.eu` or per-host certs.

Backend (separate repo / plan — the real boundary):

- ☐ `authorize("admin")` / `authorize("teacher")` middleware on route groups.
- ☐ (Optional) per-portal `aud` claim in JWT + verification.

Result: Two separately-branded domains with separate access, from one codebase, with cross-access actually closed.

Companion document: [FRONTEND_CODE_AUDIT.md](#).